Certified Reuse and Learning for Network Utility Maximisation

Utility bounds and the computational value of algorithm selection

Kai Chi
15 October 2026

Abstract

Repeated network allocation raises two questions: can a feasible update be certified without a full solve, and can historical observations identify when extra numerical work is worthwhile? For weighted shifted-log utility with hard link capacities, a cached dual price ray gives an exact scalar water-filling problem and a valid utility-loss certificate. A sharp Jensen bound explains uniform capacity contractions; an inexact-anchor extension exposes stationarity and slack errors. A two-link construction shows that an optimal repaired allocation can nevertheless have an arbitrarily loose ray certificate. On six public topology structures, the original precheck certifies 216 of 576 core updates at a normalised gap of 0.001. Its median paired runtime ratio to a strong ray warm start is 0.95, but is 1.03 for nonuniform changes. A bounded price correction and cost-sensitive selector are then evaluated with nested topology-held-out validation on 768 updates. A frozen-policy follow-up adds four unused graphs and 256 fresh updates under one and four configured BLAS threads on the same CPU. Learning does not consistently improve on a simple capacity rule: follow-up aggregate runtime ratios are 1.108 and 1.115. The central result is a separation between mathematical validity, certificate coverage and net computational value. Public graph structures, synthetic workloads and limited execution environments constrain external inference.

1 Research question and contribution

A capacity change need not require a full reoptimisation. A repaired previous allocation may already have acceptable utility, but feasibility alone does not reveal its loss. This study first constructs an inexpensive acceptance test and then treats the choice of further computation as a statistical decision. The acceptance target remains explicit throughout, including when a learned policy chooses the update action.

The analytical component connects shift heterogeneity to scaling regret and identifies a structural weakness of one-dimensional price reuse. The empirical component tests whether a small numerical correction is useful and whether a learned selector adds value over inexpensive alternatives. These are separate claims: a valid bound can accept few cases, and an accurate-looking selector can cost more than the computation it avoids.

The starting NUM work uses smooth penalties and accelerated gradients [1]. The present hard-capacity problem uses standard Lagrangian duality [2], known iso-elastic scaling [3] and water filling [4]. Dynamic multiplier adaptation [5], scalable NUM methods [6] and algorithm selection [7] provide the relevant context. No new general-purpose solver or publication priority is claimed.

2 Model and certified price reuse

Let A be a nonnegative link-route matrix with m rows and d nonempty columns. Capacities and utility weights are positive; shifts are nonnegative. Routes, weights and shifts remain fixed during an update. Natural logarithms are used. A subscript zero denotes nominal quantities; unsubscripted capacities denote the updated problem.

$$V(c) = \max_{x \geq 0,\, Ax \leq c} U(x), U(x) = \sum_i w_i \log(x_i + \xi_i), W = \sum_i w_i.$$

Every flow is bounded on its route. A strictly positive feasible point exists, and strict concavity gives a unique optimum. A zero shift requires a positive rate for finite utility; positive shifts allow optimal zero rates. For nonnegative link prices with positive route prices, maximising the separable Lagrangian gives

$$x_i(q) = (w_i / q - \xi_i)_+, \phi_i(q) = w_i \log(x_i(q) + \xi_i) - q\, x_i(q),$$
$$B(c, \lambda) = c^\top \lambda + \sum_i \phi_i(q_i), q = A^\top \lambda > 0.$$

Utility guarantee

For any feasible allocation, adding price-weighted unused capacity cannot reduce utility. Maximising over nonnegative rates therefore gives the dual upper bound B. Weak duality yields the acceptance rule

$$0 \leq V(c) - U(x) \leq B(c, \lambda) - U(x), \frac{B(c, \lambda) - U(x)}{W} \leq \epsilon.$$

The primary target is 0.001. It guarantees aggregate weighted log utility, equivalently a weighted geometric mean of shifted-rate ratios to the updated optimum of at least approximately 0.999. It does not guarantee every flow an individual throughput ratio. The implementation evaluates feasible primal and dual values in Float64 with feasibility guards; it is not an interval-arithmetic proof.

Exact scalar query and feasible repair

Restrict prices to positive multiples of the cached nominal vector. Differentiating the dual along this ray gives a unique minimising scale, characterised by

$$\gamma = \sum_i w_i (b - \alpha_i)_+, \gamma = c^\top \lambda_0, \alpha_i = \xi_i q_{0i} / w_i, b = 1/t.$$

Sorting the alpha values and caching weighted prefix sums identifies the active set by binary search. Preparing the cache costs O(nnz(A) + d log d); a new capacity-price product costs O(m), and the scalar query costs O(log d). Constructing and checking an allocation still costs O(nnz(A) + d + m). The complete online decision is not logarithmic time.

For proposed rates v, one-pass routewise downscaling ensures feasibility:

$$\ell = A v, r_e = \min\{1, c_e / \ell_e\}, x_i = v_i \min_{e : A_{ei} > 0} r_e.$$

A zero-load link uses factor one. Each flow crossing a link is reduced by at least that link's required factor. The reuse query repairs both the nominal and ray-stationary allocations, retains the higher utility, and checks it against the best ray dual value. If the target fails, the common L-BFGS-B solver starts from these prices and the retained feasible lower bound.

3 Scaling bounds and limits of the price ray

Uniform contraction

Assume the nominal allocation is an exact optimum with all rates positive and its prices satisfy KKT. For a uniform capacity contraction with factor s strictly positive and at most one, define weighted shift fractions and transformed values by

$$p_i = w_i / W\,,\theta_i = \frac{\xi_i}{x_{0i}+\xi_i}\,, z_i = s + (1-s)\theta_i\,, \bar{z} = E_p\, z_i\,.$$

Uniformly scaling the nominal allocation remains feasible, and its normalised regret satisfies

$$\frac{V(s\,c_0) - U(s\,x_0)}{W} \le J_s = \log \bar{z} - E_p \log z_i \le \frac{(1-s)^2}{2\,s^2}\, Var_p(\theta)\,.$$

Proof. KKT stationarity expresses route prices through nominal shifted rates. Extending each coordinate supremum above minus its shift gives a relaxed ray dual. Scalar minimisation bounds utility by nominal utility plus total weight times the log of the transformed mean. Subtracting scaled utility gives the Jensen gap; Taylor's inequality gives its variance bound.

Equal shift fractions give zero regret; unshifted proportional fairness is a known case [3]. On one link, set each nominal shifted rate equal to its weight, with shift fractions below their transformed mean. Nominal price one satisfies KKT and the updated optimum attains the bound. Nine saved constructions agree within 1e-12.

Approximate nominal solutions

Exact KKT assumptions are unnecessary for an operational bound. Take any feasible nominal allocation of finite utility and any nonnegative nominal prices with positive route prices. Zero nominal rates are allowed when their shifts are positive. Set

$$r_i = \frac{q_{0i}(x_{0i}+\xi_i)}{w_i}\,, \sigma = \frac{\lambda_0^{\top}(c_0 - A\,x_0)}{W}\,, a_i = r_i\, z_i\,, \bar{a} = E_p\, a_i\,.$$

$$\frac{V(s\,c_0) - U(s\,x_0)}{W} \le \log \bar{a} - E_p \log a_i + \log(1 + s\,\sigma / \bar{a})\,.$$

Minimising the relaxed ray dual proves the bound. The Jensen gap measures stationarity-adjusted shift dispersion; the last term measures price-weighted unused capacity. Exact positive-rate KKT conditions recover the earlier bound. In 500 checks, the expression matches explicit relaxed-dual evaluation to 1.4e-15; the exact nonnegative-rate ray certificate is at least as strong.

An optimal allocation can still fail certification

Take two disjoint links, unit weights, zero shifts, nominal capacities (1,1) and nominal prices (1,1). Reducing the first capacity to r makes the repaired allocation (r,1) exactly optimal, yet

$$\frac{\min_{t>0} B((r,1), t(1,1)) - U((r,1))}{2} = \frac{1}{2}\log\frac{(1+r)^2}{4\,r} \to \infty\,(r \downarrow 0)\,.$$

A rejected certificate therefore does not prove poor utility. Nonuniform changes may require rotation of prices, which a scalar ray cannot express. This motivates correcting prices and candidates together, while retaining a valid gap as the acceptance criterion.

4 Numerical refinement and learned action selection

A bounded correction before fallback

The original and refinement actions share the initial ray query. An already certified case returns immediately. Otherwise the original action invokes the common solver; refinement first proposes componentwise capacity-ratio prices:

$$\lambda_e^{ratio} = max\{10^{-12}, \lambda_{0,e} c_{0,e} / c_e\}.$$

This predictor is exact for independent links with unshifted logs at an exact nominal solution, including the two-link example. It is a heuristic on coupled networks. The method then attempts at most two safeguarded Newton corrections on at most eight price coordinates, ranked by absolute dual gradient divided by the square root of the regularised Hessian diagonal.

Within a fixed rate active set, the reduced Hessian uses the selected routing rows, weighted by utility weight divided by squared route price on positive-rate coordinates. The linear system uses a ridge of 1e-8 times the maximum diagonal with a positive guard. Prices have a 1e-12 floor; an Armijo search tries at most eight step lengths. Every trial allocation is repaired before its utility is retained. Active sets may change, so safeguarding is required.

The smallest valid dual value and largest feasible utility are retained independently across steps. Their difference remains a valid upper bound on regret and cannot increase when valid incumbents are added. If the target still fails, the shared L-BFGS-B fallback runs. A failed fallback is recorded as a failure and never accepted as certified output.

Learning when the correction is worthwhile

Only the 360 core cases rejected by the initial certificate supply fitting examples. Each action's historical runtime is reduced to a median of three observations. The regression target and decision are

$$y(z) = \log\left(T_{refine} / T_{reuse}\right), \pi(z) = 1_{\{f(z) < \tau\}}, \tau \in \{0, -0.05\}.$$

Sixteen features describe problem size, mean route length, nominal shift fractions, capacity ratios, price-weighted changes, active nominal prices and the initial certificate. Graph names, identifiers, shock labels, updated reference solutions and current-case action runtimes are excluded. Feature extraction is skipped for initially certified cases.

Candidates are ridge regression with penalties 1 and 10, and trees of depth 1, 2 or 3 with at least 20 examples per leaf. Each outer fold excludes one entire topology, including all its seeds and settings. An inner leave-one-topology-out loop on the remaining five graphs chooses the model and threshold. Scaling is fitted within each training split. The inner score gives equal weight to each validation graph and charges measured feature cost; prediction cost is charged in the final replay.

Each outer fold has 300 fitting examples and evaluates all 96 held-out core cases. A portable fitted model is evaluated directly from stored coefficients or tree arrays. The training-fixed baseline is selected using training graphs alone. The later fresh-graph test freezes the previously exported model fitted on all six original graphs; none of its outcomes are used to refit or retune that model.

5 Data and measurement protocol

The original graph structures are Abilene, Geant2012, Cernet, Colt, UsCarrier and Cogentco. Four additional structures are Dfn, Garr201201, Surfnet and BtEurope. All come from Internet Topology Zoo [8], pinned to commit 51118849b99534f01010d885d66321433a7eef9e. Self loops and parallel edges are removed, the largest component retained and edges expanded into directed links. The original GraphML files, hashes and CC BY 4.0 attribution are retained.

Only graph structures are observed. Shortest-path routes, lognormal mean-one weights, capacities and shocks are generated. The core design crosses three seeds, 1,000 or 10,000 flows, load-coupled or independent capacities, and shifts zero or 0.5. Four shocks per nominal anchor give 576 core updates: uniform reductions of 10% or 30%, a 30% reduction on a random 5% of used links, and heterogeneous reductions up to 30%.

Measurement phase	Capacity cases	Methods and repetitions	Timing records
Original NUM benchmark	576 core + 24 scaling	4 methods x 3	7,200
Historical action collection	576 core	2 actions x 3	3,456
Topology-held-out replay	576 core + 192 shifted	6 pipelines x 3	13,824
Frozen fresh-graph replay	256 new	6 x 3 x 2 BLAS limits	9,216

The 192 shifted cases reuse 48 original nominal anchors but change shock severity: local cuts of 5% or 50% on 5% of links, heterogeneous cuts up to 60%, or capacity factors from 0.8 to 1.2. They are new shocks rather than independent new networks. The original scaling extension uses 24 cases on two graphs up to 100,000 flows; no learning result is claimed at that size.

Fresh-graph validation uses route seeds 1701 and 1907, both flow counts, both capacity laws and both shifts: 64 new anchors and four updates each. Shocks are uniform 10%, local 30%, heterogeneous up to 30%, and factors from 0.8 to 1.2. Its policy, graph identities and settings were fixed before generating new outcomes, informed by earlier findings. This is not blinded or externally preregistered replication.

Comparable online costs

All methods use the same feasible-primal/dual target of 0.001. Tight nominal and updated references target 1e-7; fallback caps are 3,000 iterations and 45,000 evaluations. Methods rotate within repetitions. Timers include features, prediction, repair, certificates, corrections, line searches and fallback. Data loading and offline fitting are excluded. Repeated timings measure execution variation, not independent network draws.

NUM runtime summaries charge cache preparation amortised over four core updates; anchor-inclusive sensitivity is reported separately. Learning comparisons use complete online costs and record cache preparation separately. Measurements from different phases are not pooled into an absolute speedup estimate. The learning phase verifies one BLAS thread; fresh-graph tests use limits one and four on the same CPU, checking both NumPy and SciPy pools. This tests configuration sensitivity rather than different hardware. An interrupted collection with inconsistent pool limits remains archived and is excluded from analysis.

6 Certificate coverage and the original NUM benchmark

The original precheck certifies 216 of 576 core cases, or 37.5%, without iterative optimisation. Every acceptance follows a uniform contraction. All 144 unshifted uniform cases pass, as expected from proportional-fair scaling; the remaining 72 acceptances have positive shifts.

Capacity change	Certified by ray	Cases
Uniform 10% reduction	144	144
Uniform 30% reduction	72	144
Local 30% reduction	0	144
Heterogeneous up to 30%	0	144

The unchanged-price certificate accepts none. A controlled two-by-two ablation holds the candidate fixed while changing prices, then holds prices fixed while changing the candidate. Both repaired candidates give the same acceptance counts: zero at nominal prices and 216 at the best ray prices. These controls isolate the dual improvement for this dataset; they do not establish that candidate choice never matters.

Tight reference intervals identify the same 216 candidates as within tolerance. No reference-good core candidate is missed; the other 360 have a reference lower bound on regret above 0.001. Thus the counterexample identifies a structural possibility, while this benchmark principally needs better allocations. A stronger upper bound alone cannot make those 360 candidates meet the target.

Method	Solved core cases	Median cost ms	Median FG calls
Cold prices	576 / 576	44.20	66.5
Warm prices	576 / 576	23.49	33
Ray warm start	576 / 576	9.36	21
Certified reuse	576 / 576	9.63	21

All four methods share L-BFGS-B and the same target; even cold prices receive a repaired nominal primal allocation. Median paired reuse-to-baseline ratios are 0.45 for cold prices, 0.83 for warm prices and 0.95 for ray warm start. The last has a descriptive topology-block interval [0.94, 0.97]. A median of paired ratios differs from a ratio of marginal medians, explaining why the table alone does not recover that result.

Excluding the known unshifted uniform cases gives a reuse-to-ray-warm median ratio of 1.00 on 432 cases. Restricting to local and heterogeneous changes gives 1.03 on 288 cases. Adding the nominal solve amortised over four updates changes the all-core ratio from 0.95 to 1.00. The useful speed advantage is therefore concentrated in a tractable regime and depends on accounting for reusable information.

The 24-case scaling extension is descriptive, not a scaling law. All four methods miss the target on the unshifted 100,000-flow Cogentco heterogeneous case in all three repetitions: 12 failed runs. Their capped times remain in the results. Original reference solves meet their target. No GPU or modern matched-model solver comparison was performed.

7 Does learning improve the update decision

Six pipelines are replayed on 576 core and 192 shifted cases. Ray warm start directly solves from scalar-adjusted prices. Original reuse applies the shared query then falls back. Always refine performs the bounded correction after rejection. The capacity rule refines when the standard deviation of elementwise updated-to-nominal capacity ratios exceeds 1e-4. Training fixed and learned selection use only permitted training information.

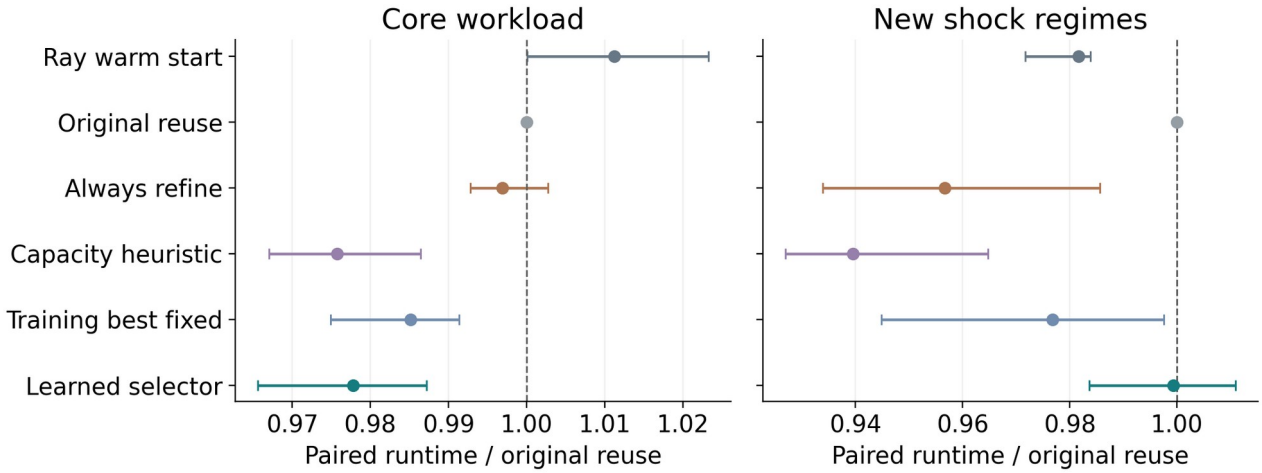


Figure 1. Median paired runtime ratios to original reuse, with descriptive topology-block bootstrap intervals. Lower values favour the displayed pipeline. All online selection and correction costs are included.

Pipeline	Core median	Shift median	Aggregate core / shift
Ray warm start	1.011	0.982	0.996 / 0.980
Original reuse	1.000	1.000	1.000 / 1.000
Always refine	0.997	0.957	0.842 / 0.849
Capacity rule	0.976	0.940	0.835 / 0.842
Training fixed	0.985	0.977	0.910 / 0.877
Learned selector	0.978	0.999	0.836 / 0.904

Aggregate ratios compare sums of case-median times; paired medians compare the typical case. The capacity rule matches the learner's core aggregate cost and performs better under shifted shocks. Against the training-fixed action, the learner's median ratios are 0.992 and 1.013. Learning does not show a consistent advantage over these inexpensive selectors.

Initial certification accepts 216 core and 5 shifted cases. Always refine adds 10 and 17 early certifications; the learned policy adds 9 and 10. These are intermediate outcomes rather than speed guarantees. Every pipeline solves all core cases and 191 of 192 shifted cases. All 18 failed replay runs arise from the same UsCarrier 10,000-flow heterogeneous-60% case, with final gaps 0.00249 to 0.00310. Capped times remain in the primary summaries and no failed output is accepted.

Among cases rejected by the initial certificate, the learned-to-ray-warm core median ratio is 1.015. The favourable all-core comparison therefore also reflects shared early certification, not just learned decisions. The six graph groups and reused anchors limit inference; the intervals describe this collection rather than population coverage.

8 Frozen policy validation on four unused graphs

The follow-up freezes the previously exported all-core model and tests fresh nominal problems on four unused graph identities. After preprocessing, DFN, GARR 201201, SURFnet and BT Europe have 58, 61, 50 and 24 nodes, with 87, 75, 68 and 37 undirected edges. All 256 cases are certified by every pipeline in all repetitions under both numerical-library configurations; there are no failed follow-up runs.

Pipeline	Median 1 thread	Median 4 threads	Aggregate 1 / 4
Ray warm start	1.156	1.161	1.333 / 1.353
Original reuse	1.016	1.023	1.352 / 1.355
Always refine	1.003	0.997	0.995 / 0.994
Capacity rule	1.000	1.000	1.000 / 1.000
Training fixed	0.995	0.990	0.986 / 1.019
Learned selector	1.000	0.988	1.108 / 1.115

Ratios use the capacity rule as baseline. The learner's paired medians are 1.000 and 0.988, while aggregate ratios are 1.108 and 1.115: aggregate cost increases by 10.8% and 11.5%. Win fractions are 49.6% and 56.6%; 90th-percentile ratios are 1.199 and 1.244. At four threads, a favourable typical-case result coexists with a costly slower tail.

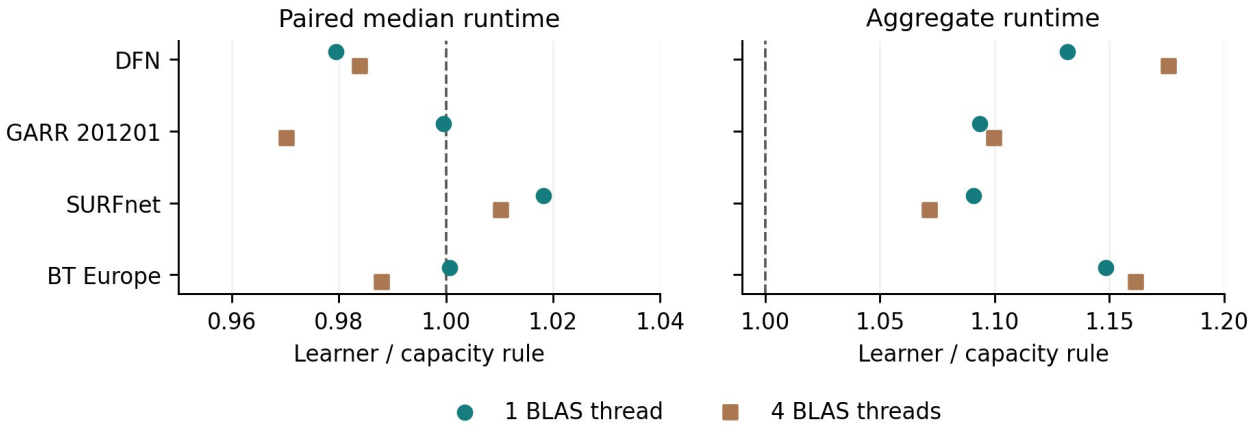


Figure 2. Learner-to-rule ratios within each fresh graph. Paired medians and aggregate costs answer different questions. All four graphs have aggregate ratios above one under both configurations; the aggregate loss is not confined to one graph.

At 1,000 flows, learner-to-rule median ratios are 0.983 and 0.976 under one and four threads; at 10,000 flows they are 1.009 and 0.999. Larger instances therefore do not automatically create a stable selection benefit. The two BLAS limits use the same CPU and software, and not every operation parallelises.

These graphs are convenience-selected historical structures from the same source collection. Cases within a graph share structure and some routes. Three repetitions add timing observations rather than independent networks. No population interval is inferred from four graphs, and the unchanged workload generator limits the meaning of external transfer.

9 Selection cost and numerical audit

A retrospective diagnostic asks how much additional selection work could be worthwhile. For each rejected case, take the smaller observed median time of original reuse and always refine. Subtract it from the capacity-rule time and clip at zero. This hindsight budget uses test timings after evaluation; it is never an online input, and the observed minimum can be optimistic under timing noise.

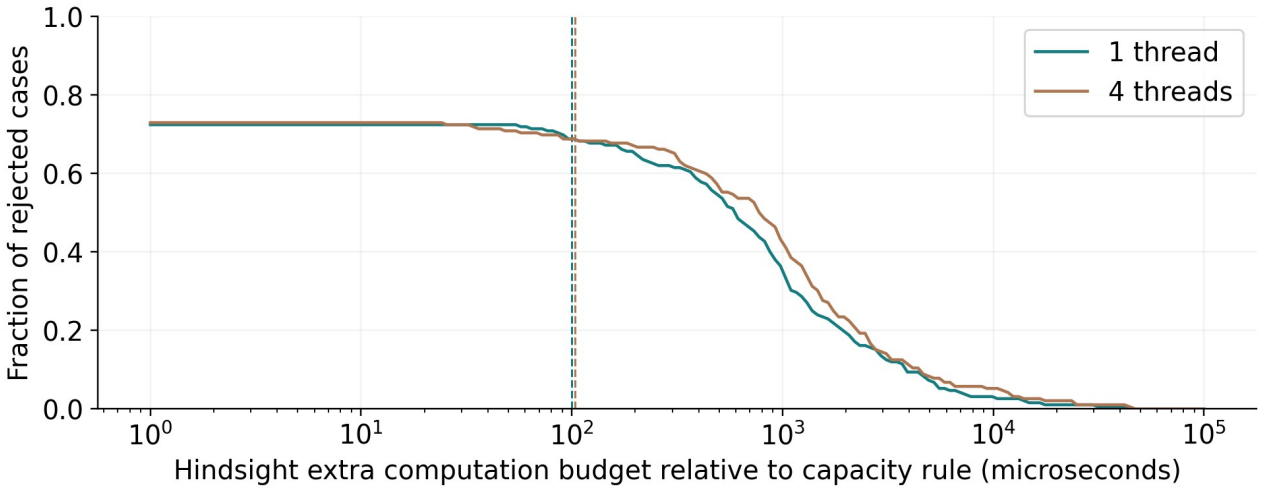


Figure 3. Fraction of rejected fresh-graph cases with at least the displayed hindsight budget. Dashed lines show measured median feature-plus-prediction cost. This is a retrospective timing diagnostic, not a deployable oracle or a bound on achievable expected speed.

BLAS threads	Gate cost us	Budget us	Budget covers gate	Action disagreement
1	100.0	589.2	69.3%	40.1%
4	103.8	773.4	68.8%	37.5%

Each configuration has 192 initially rejected cases. Costs and budgets in the table are medians. The hindsight best-of-two aggregate ratios to the rule are 0.957 and 0.955, before additional selection cost. The learner disagrees with the observed cheaper action on 40.1% and 37.5% of rejected cases. These quantities separate observed opportunity, action ranking and gate cost; they do not show that available online features can recover a hindsight oracle.

Median budgets exceed median gate costs, and about 69% of rejected cases have budgets covering their measured gate cost. Inference overhead alone therefore does not describe the whole difficulty. Transfer of action-cost relationships and expensive decisions also deserve investigation. The present diagnostic does not isolate their causal contributions.

Checks against stored numerical evidence

The original audit recomputes 600 stored problems, with maximum ray-gap discrepancy 1e-16 and relative candidate overload about 1.1e-15. Independent scalar minimisation, small primal SLSQP solves and finite-anchor checks support the implementation. The learning audit reconstructs 1,152 historical action outputs and 768 held-out policy decisions. The fresh-graph audit recomputes 3,072 saved primal/dual outputs and 512 configuration-specific decisions; its maximum saved-gap discrepancy is about 1e-16 and observed overload is zero.

Source and instance hashes, graph separation, frozen policy identity and configured BLAS limits also pass their checks.

10 Conclusions and research directions

The mathematical and empirical results support a conditional conclusion. Cached prices can certify some updates cheaply, and the shifted-log bound explains why uniform changes are favourable. Nonuniform changes expose both candidate-quality and price-direction limitations. A bounded correction can reduce aggregate computation, but a fitted selector does not consistently improve on a simple rule once its complete online cost is charged.

The next useful test should combine independent workload provenance with a genuinely different machine, keep policies frozen and retain inexpensive baselines. Sequential capacity changes would allow cache refresh, retraining and preparation to be charged over an observed reuse horizon. Refitting on the present test outcomes would require another untouched test collection. A matched-model modern solver comparison is also needed before making solver-superiority claims.

The study uses fixed routes, synthetic capacities and shocks, ten convenience graph identities and limited execution environments. It omits production traffic, multipath decisions, demand arrivals and controller communication. The supplied code-and-data archive retains the original arrays, raw timings, failures, split manifests, fitted models, proofs, provenance and analysis scripts. The measured results establish a reproducible case study of computational decisions, with explicit limits on generalisation.

References

[1] Y. Tian et al. Centralized Network Utility Maximization with Accelerated Gradient Method. IEEE ICNP, 2022; author preprint arXiv:2408.08034. Source

[2] S. Boyd and L. Vandenberghe. Convex Optimization. Cambridge University Press, 2004. Source

[3] S. Borst, N. Walton and B. Zwart. Network iso-elasticity and weighted alpha-fairness. arXiv:1201.2292, 2012. Source

[4] D. P. Palomar and J. R. Fonollosa. Practical Algorithms for a Family of Waterfilling Solutions. IEEE Transactions on Signal Processing 53(2), 686-695, 2005. Source

[5] I.-H. Hou. Network Optimization in Dynamic Systems: Fast Adaptation via Zero-Shot Lagrangian Update. arXiv:2412.07865; accepted for IEEE INFOCOM 2025. Source

[6] A. Sreekumar, A. Degleris and R. Rajagopal. Large-Scale Network Utility Maximization via GPU-Accelerated Proximal Message Passing. IEEE Control Systems Letters 9, 2807-2812, 2025; author version arXiv:2509.10722v2, 2026. Source

[7] B. Bischl et al. ASlib: A Benchmark Library for Algorithm Selection. Artificial Intelligence 237, 41-58, 2016. Source

[8] S. Knight et al. The Internet Topology Zoo. IEEE Journal on Selected Areas in Communications 29(9), 1765-1775, 2011. Source